

Assignment: Stock Market Data Aggregation Service

Overview

Build a RESTful service (in **Java** or **Golang**) that reads stock market time-series data from Apache Cassandra and returns aggregated OHLCV (Open, High, Low, Close, Volume) candlestick data converted to a requested timeframe, suitable for charting.

Prerequisites

- Apache Cassandra (installed and running locally)
- Java with Spring Boot **OR** Golang
- Provided CSV file: stock_data.csv

Part 1: Database Setup (One-Time Activity)

1.1 Install Apache Cassandra Locally

- Install and configure Apache Cassandra on your local machine.
- Ensure the instance is running and accessible on the default port (9042).

1.2 Schema Design

- Create a keyspace (e.g., stock_keyspace) with an appropriate replication strategy for local development.
- Design and create a table to store the stock data optimally for time-series queries.

CSV columns provided:

Column	Description	Example
symbol	Stock ticker symbol	RELIANCE
datetime	Timestamp of the candle	2024-01-15 09:15:00
open	Opening price	2450.50

Column	Description	Example
high	Highest price in the interval	2462.20
low	Lowest price in the interval	2448.10
close	Closing price	2458.90
volume	Number of shares traded	125000

Note: The provided data is in **1-minute** candle intervals.

1.3 Data Ingestion

- Write a script or utility to parse the CSV and insert all records into the Cassandra table.
- This is a one-time activity and can be a separate script/module.

Part 2: Service Development

2.1 API Endpoint

Build a REST API endpoint that accepts the following parameters and returns aggregated candlestick data:

Endpoint:

GET /api/v1/candles

Query Parameters:

Parameter	Type	Required	Description	Example
symbol	string	Yes	Stock ticker symbol	RELIANCE
timeframe	string	Yes	Target aggregation timeframe	5m, 15m, 1h, 1d
start_date	string	Yes	Start datetime (ISO 8601 / yyyy-MM-dd HH:mm:ss)	2024-01-15 09:15:00

Parameter	Type	Required	Description	Example
end_date	string	Yes	End datetime (ISO 8601 / yyyy-MM-dd HH:mm:ss)	2024-01-15 15:30:00

Supported Timeframes:

Code	Meaning
1m	1 Minute (raw data)
5m	5 Minutes
15m	15 Minutes
30m	30 Minutes
1h	1 Hour
1d	1 Day

2.2 Aggregation Logic

When converting from 1-minute candles to a higher timeframe, apply the following rules:

- **Open:** First open value in the aggregation window
- **High:** Maximum high value across all candles in the window
- **Low:** Minimum low value across all candles in the window
- **Close:** Last close value in the aggregation window
- **Volume:** Sum of all volume values in the window

2.3 Response Format

Return the data in a JSON format suitable for charting libraries (e.g., TradingView, Chart.js, Highcharts):

```
{
  "symbol": "RELIANCE",
  "timeframe": "15m",
  "candles": [
```

```
{
  "datetime": "2024-01-15T09:15:00Z",
  "open": 2450.50,
  "high": 2472.30,
  "low": 2448.10,
  "close": 2465.80,
  "volume": 540000
},
{
  "datetime": "2024-01-15T09:30:00Z",
  "open": 2465.80,
  "high": 2480.00,
  "low": 2463.50,
  "close": 2475.60,
  "volume": 430000
}
],
"count": 2
}
```

Part 3: Client Application

3.1 Backend Client (Required)

Create a **separate project** that acts as a client application for the service built in Part 2.

Requirements:

- The client should consume the candle aggregation API built above.
- Accept input parameters (symbol, timeframe, start_date, end_date) via command-line arguments or a configuration file.
- Make an HTTP call to the service endpoint.
- Parse the JSON response and **log the output** in a clean, readable format to the console.

Example log output:

=== Fetched Candle Data ===

Symbol: RELIANCE | Timeframe: 15m | Total Candles: 2

¹ 2024-01-15T09:15:00Z | O: 2450.50 | H: 2472.30 | L: 2448.10 | C: 2465.80 | V: 540000

² 2024-01-15T09:30:00Z | O: 2465.80 | H: 2480.00 | L: 2463.50 | C: 2475.60 | V: 430000

=====

This is to evaluate your ability to consume RESTful APIs and handle HTTP communication between services.

3.2 Frontend Web Client (Bonus — Added Advantage)

If you have frontend experience, build a **Vue.js** web application that:

- Provides a simple UI with input fields for symbol, timeframe, start date, and end date.
- Calls the candle aggregation API on form submission.
- Renders the returned data as a **candlestick chart** (using a charting library such as Highcharts).

Note: This is entirely optional and meant as an added advantage for candidates with full-stack capabilities. It will NOT negatively impact your evaluation if skipped.

Part 4: Requirements & Evaluation Criteria

Functional Requirements

- Cassandra is set up locally with appropriate schema design
- CSV data is ingested correctly into Cassandra
- API endpoint correctly accepts all required parameters
- Timeframe aggregation logic is accurate
- Response is returned in the specified chart-friendly JSON format
- Proper error handling for invalid inputs (missing params, unsupported timeframe, invalid date range, symbol not found)
- Client application successfully consumes the API and logs output

Non-Functional Requirements

- Clean, well-structured, and readable code
- Proper separation of concerns (controller/handler, service, repository layers)

- Efficient Cassandra queries (leveraging partition keys and clustering columns)
- Input validation and meaningful error responses with appropriate HTTP status codes
- README with setup instructions, assumptions, and how to run the project

Bonus Points

- Unit tests for the aggregation logic
- Pagination support for large date ranges
- Caching frequently requested timeframe conversions
- API documentation
- Logging and basic observability
- Vue.js frontend web client with candlestick chart visualization

Submission Guidelines

1. Organize your project in a clean folder structure:
2. submission/
3. └─ server/ # Main API service
4. └─ client/ # Backend client application
5. └─ web-client/ # (Optional) Vue.js frontend
6. └─ schema.cql # Cassandra schema file
7. └─ README.md # Setup & run instructions
8. Include a README.md with:
 - Setup and installation instructions
 - How to run the data ingestion script
 - How to start the service
 - How to run the client application
 - Sample cURL/HTTP requests and responses
 - Any assumptions or design decisions made
9. Include the CQL schema file (schema.cql) separately for easy review.
10. Compress your entire project folder into a .zip file and send it to assignments@definededge.com from the same email address you provided in the form (If you haven't filled the form yet, please do so here: [Backend Developer - Profile & Skill Assessment – Fill out form](#)). Use the email subject line: Assignment Submission — [Your Full Name]